

ผลการทดสอบ Endpoint

GET เพื่อดูเมนูทั้งหมด

curl http://localhost:8080/coffees

```
D:\JNP @KKU\3 term 1\Software Design\Lab 5\LAB5-673380299-1\demo>curl http://localhost:8080/coffees
[{"id":1,"name":"Espresso","price":55.0},{ "id":2,"name":"Latte","price":70.0},{ "id":3,"name":"Mocha","price":80.0}]
```

GET by id เพื่อดูเมนู 1 รายการตาม id

curl http://localhost:8080/coffees/2

```
D:\JNP @KKU\3 term 1\Software Design\Lab 5\LAB5-673380299-1\demo>curl http://localhost:8080/coffees/2
{"id":2,"name":"Latte","price":70.0}
```

POST เพื่อเพิ่มเมนูใหม่ ด้วยคำสั่ง

curl.exe -X POST http://localhost:8080/coffees -H "Content-Type: application/json" --data-binary "@newMenu.json"

```
D:\JNP @KKU\3 term 1\Software Design\Lab 5\LAB5-673380299-1\demo>curl.exe -X POST http://localhost:8080/coffees -H "Content-Type: application/json" --data-binary "@newMenu.json"
Coffee Added
```

จากภาพจะเห็นว่ามีความ Coffee Added ปรากฏ ซึ่งหมายถึงเพิ่มเมนูใหม่ตามที่กำหนดใน newMenu.json เรียบร้อยแล้ว

```
D:\JNP @KKU\3 term 1\Software Design\Lab 5\LAB5-673380299-1\demo>curl http://localhost:8080/coffees
[{"id":1,"name":"Espresso","price":55.0},{ "id":2,"name":"Latte","price":70.0},{ "id":3,"name":"Mocha","price":80.0},{ "id":4,"name":"Americano","price":75.0}]
```

และเมื่อตรวจสอบเมนูทั้งหมดจะเห็นได้ว่ามีเมนูที่เพิ่มขึ้นมาใหม่อยู่ในลิสต์ คือ Americano

PUT เพื่อแก้ไขเมนูเดิมตาม id ด้วยคำสั่ง

curl.exe -X PUT http://localhost:8080/coffees/4 -H "Content-Type: application/json" --data-binary "@updateMenu.json"

```
D:\JNP @KKU\3 term 1\Software Design\Lab 5\LAB5-673380299-1\demo>curl.exe -X PUT http://localhost:8080/coffees/4 -H "Content-Type: application/json" --data-binary "@updateMenu.json"
Updated
D:\JNP @KKU\3 term 1\Software Design\Lab 5\LAB5-673380299-1\demo>curl http://localhost:8080/coffees/4
{"id":4,"name":"Iced Americano","price":80.0}
```

จากภาพจะเห็นได้ว่าเมนู Americano ได้ถูกเปลี่ยนชื่อเป็น Iced Americano และราคาถูกเปลี่ยนจาก 75 เป็น 80 ตามที่กำหนดในไฟล์ updateMenu.json

DELETE เพื่อลบเมนูตาม id ด้วยคำสั่ง

curl.exe -X DELETE http://localhost:8080/coffees/4

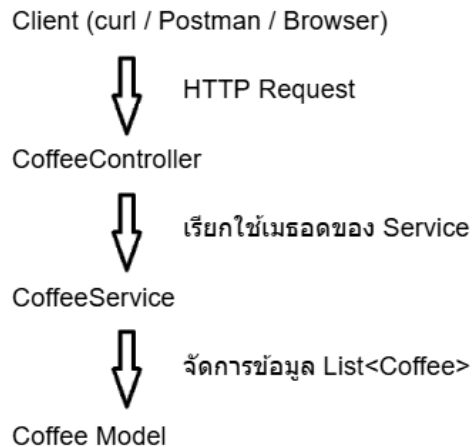
```
D:\JNP @KKU\3 term 1\Software Design\Lab 5\LAB5-673380299-1\demo>curl.exe -X DELETE http://localhost:8080/coffees/4
Deleted
D:\JNP @KKU\3 term 1\Software Design\Lab 5\LAB5-673380299-1\demo>curl http://localhost:8080/coffees
[{"id":1,"name":"Espresso","price":55.0},{ "id":2,"name":"Latte","price":70.0},{ "id":3,"name":"Mocha","price":80.0}]
D:\JNP @KKU\3 term 1\Software Design\Lab 5\LAB5-673380299-1\demo>curl http://localhost:8080/coffees/4
D:\JNP @KKU\3 term 1\Software Design\Lab 5\LAB5-673380299-1\demo>
```

จากภาพจะเห็นว่าหลังใช้คำสั่ง DELETE จะไม่พบเมนู Iced Americano อยู่ในลิสต์แล้ว และเมื่อตรวจสอบด้วยการหาตาม id ก็จะไม่พบเช่นกัน

ตอบคำถาม Discussion

1. HTTP method แต่ละตัว (GET/POST/PUT/DELETE) ต่างกันอย่างไร ยกตัวอย่างจากโปรเจกต์ตัวเอง

ตอบ การทำงานของ HTTP method แต่ละตัวมีหน้าที่แตกต่างกัน โดยมีทำงานร่วมกับ Controller และ Service ซึ่งสามารถเขียนเป็นลำดับได้ดังนี้



- GET ใช้สำหรับอ่านข้อมูลจาก Server โดยจะไม่มี การเปลี่ยนแปลงข้อมูลในระบบ เมื่อผู้ใช้ส่ง Request GET /coffees หรือ GET /coffees/{id} Controller จะรับคำขอ GET และเรียก getAllCoffee(); หรือ getCoffee(id) ผ่าน Service

Controller

```

@GetMapping
public List<Coffee> getAllCoffee() {
    return service.getAllCoffee();
}

@GetMapping("/{id}")
public Coffee getCoffee(@PathVariable int id) {
    return service.getCoffee(id);
}
  
```

ภายใน Service มีลิสต์ (List<Coffee>) ซึ่งเก็บเมนูทั้งหมดเอาไว้ และ method GET เพื่อคืนข้อมูลกลับไปให้ Controller

```

private List<Coffee> coffeeList = new ArrayList<>();
  
```

```
// GET
public List<Coffee> getAllCoffee() {
    return coffeeList;
}

// GET by id
public Coffee getCoffee(int id) {
    for (Coffee coffee : coffeeList) {
        if (coffee.getId() == id) {
            return coffee;
        }
    }
    return null;
}
```

- POST ใช้สำหรับเพิ่มข้อมูล
เมื่อผู้ใช้ส่ง Request POST /coffees Controller จะรับไฟล์ JSON แล้วแปลงไฟล์ JSON เป็น Coffee Object จากนั้นจึง ส่ง Coffee ให้ Service

Controller

```
@PostMapping
public String addCoffee(@RequestBody Coffee coffee) {
    service.addCoffee(coffee);
    return "Coffee Added";
}
```

Service จะเป็นผู้เพิ่มข้อมูลลงในลิสต์

```
// POST
public void addCoffee(Coffee coffee) {
    coffeeList.add(coffee);
}
```

- PUT ใช้แก้ไขข้อมูล
เมื่อผู้ใช้ส่ง Request PUT /coffees/{id} Controller จะรับ id จาก url และ ไฟล์ JSON ที่ส่งมา จากนั้นจึงส่ง id และ Coffee Object ไปให้ Service

Controller

```
@PutMapping("/{id}")
public String updateCoffee(@PathVariable int id,
                           @RequestBody Coffee coffee) {
    if (service.updateCoffee(id, coffee))
        return "Updated";

    return "Coffee Not Found";
}
```

Service จะทำการหาข้อมูลเดิมในลิสต์ หากพบจะแก้ไขข้อมูลตามที่ส่งมาและคืนค่า true แต่หากไม่พบจะคืนค่า false

```
// PUT
public boolean updateCoffee(int id, Coffee newCoffee) {

    Coffee oldCoffee = getCoffee(id);

    if (oldCoffee == null) {
        return false;
    }

    oldCoffee.setName(newCoffee.getName());
    oldCoffee.setPrice(newCoffee.getPrice());

    return true;
}
```

- DELETE ใช้ลบข้อมูล
เมื่อผู้ใช้ Request DELETE /coffees/{id} Controller จะรับ id จาก url แล้วส่งให้ Service

Controller

```
@DeleteMapping("/{id}")
public String deleteCoffee(@PathVariable int id) {

    if (service.deleteCoffee(id))
        return "Deleted";

    return "Coffee Not Found";

}
```

Service จะค้นหาข้อมูลจาก id หากพบจะลบออกลิสต์ แต่หากไม่พบจะคืนค่า false

```
// DELETE
public boolean deleteCoffee(int id) {

    Coffee coffee = getCoffee(id);

    if (coffee == null) {
        return false;
    }

    coffeeList.remove(coffee);

    return true;
}
```

2. ทำไมต้องแยก Controller กับ Service ออกจากกัน มีข้อดีอย่างไรถ้าโปรแกรมโตขึ้น

ตอบ เพราะการให้ Controller จัดการลิสต์ของเมนู (List<Coffee>) เอง จะทำให้ Controller มีหน้าที่มากเกินไป กล่าวคือทั้งรับ Request และจัดการข้อมูล ด้วยเหตุนี้จึงแยก Service เพื่อทำหน้าที่ในการจัดการข้อมูล ส่งผลให้อินเตอร์เฟซสามารถแก้ไขโค้ดได้ง่ายหากมีการเปลี่ยนไปใช้ฐานข้อมูลอื่น เช่น MySQL หรือ PostgreSQL โดยทำการแก้ไขที่ Service ส่วน Controller ยังสามารถใช้โค้ดเดิมได้

3. ข้อมูลที่เก็บไว้ใน List ใน memory หายไปตอนไหน และถ้าอยากให้ไม่หายควรทำอย่างไร (ตอบเป็นแนวคิดพอ)

ตอบ ข้อมูลจะหายทันทีเมื่อโปรแกรมหยุดทำงาน เนื่องจากหน่วยความจำ RAM จะถูกล้าง เมื่อรันโปรแกรมใหม่ CoffeeService จะสร้าง List<Coffee> ขึ้นมาใหม่จากข้อมูลเริ่มต้นใน Constructor ทำให้ข้อมูลที่เคยเพิ่มไว้ก่อนหน้านี้หายไปทั้งหมด หากไม่อยากจะหายไปควรเก็บข้อมูลไว้ในหน่วยเก็บข้อมูลแบบถาวร เช่น ฐานข้อมูล หรือไฟล์ จากนั้นจึงให้โปรแกรมมีการอ่านข้อมูลตอนเริ่มและบันทึกข้อมูลทุกครั้ง

4. @RestController, @GetMapping, @PostMapping, @PathVariable, @RequestBody แต่ละตัวทำหน้าที่อะไร

ตอบ

@RestController ใช้เพื่อกำหนดให้คลาสเป็น REST Controller ซึ่งทำหน้าที่รับ HTTP Request จาก Client และส่งผลลัพธ์กลับในรูปแบบ JSON โดย Spring จะจัดการแปลง Java Object เป็น JSON ให้อัตโนมัติ

@GetMapping ใช้เพื่อกำหนดว่าเมธอดนี้จะทำงานเมื่อมี HTTP GET Request เข้ามา ซึ่งเหมาะสำหรับการอ่านข้อมูล

@PostMapping ใช้เพื่อกำหนดว่าเมธอดนี้จะทำงานเมื่อมี HTTP POST Request เข้ามา เหมาะสำหรับการเพิ่มข้อมูลใหม่

@PathVariable ใช้รับค่าที่อยู่ใน URL มาเก็บไว้ในตัวแปรของเมธอด

@RequestBody ใช้รับข้อมูล JSON ที่ส่งมากับ HTTP Request แล้วแปลงเป็น Java Object อัตโนมัติ